

理解。例如，如果一个服务器写一个已经被客户端关闭了的连接(比如，因为你在浏览器上单击了“Stop”按钮)，那么第一次这样的写会正常返回，但是第二次写就会引起发送 SIGPIPE 信号，这个信号的默认行为就是终止这个进程。如果捕获或者忽略 SIGPIPE 信号，那么第二次写操作会返回值 -1，并将 `errno` 设置为 `EPIPE`。`strerr` 和 `perror` 函数将 `EPIPE` 错误报告为“Broken pipe”，这是一个迷惑了很多人的不太直观的信息。总的来说，一个健壮的服务器必须捕获这些 SIGPIPE 信号，并且检查 `write` 函数调用是否有 `EPIPE` 错误。

11.7 小结

每个网络应用都是基于客户端-服务器模型的。根据这个模型，一个应用是由一个服务器和一个或多个客户端组成的。服务器管理资源，以某种方式操作资源，为它的客户端提供服务。客户端-服务器模型中的基本操作是客户端-服务器事务，它是由客户端请求和跟随其后的服务器响应组成的。

客户端和服务端通过因特网这个全球网络来通信。从程序员的观点来看，我们可以把因特网看成是一个全球范围的主机集合，具有以下几个属性：1)每个因特网主机都有一个唯一的 32 位名字，称为它的 IP 地址。2)IP 地址的集合被映射为一个因特网域名的集合。3)不同因特网主机上的进程能够通过连接互相通信。

客户端和服务端通过使用套接字接口建立连接。一个套接字是连接的一个端点，连接以文件描述符的形式提供给应用程序。套接字接口提供了打开和关闭套接字描述符的函数。客户端和服务端通过读写这些描述符来实现彼此间的通信。

Web 服务器使用 HTTP 协议和它们的客户端(例如浏览器)彼此通信。浏览器向服务器请求静态或者动态的内容。对静态内容的请求是通过从服务器磁盘取得文件并把它返回给客户端来服务的。对动态内容的请求是通过在服务器上一个子进程的上下文中运行一个程序并将它的输出返回给客户端来服务的。CGI 标准提供了一组规则，来管理客户端如何将程序参数传递给服务器，服务器如何将参数以及其他信息传递给子进程，以及子进程如何将它的输出发送回客户端。只用几百行 C 代码就能实现一个简单但是有功效的 Web 服务器，它既可以提供静态内容，也可以提供动态内容。

参考文献说明

有关因特网的官方信息源被保存在一系列的可免费获取的带编号的文档中，称为 RFC(Requests for Comments，请求注解，Internet 标准(草案))。在以下网站可获得可搜索的 RFC 的索引：

<http://rfc-editor.org>

RFC 通常是因特网基础设施的开发者编写的，因此，对于普通读者来说，往往过于详细了。然而，要想获得权威信息，没有比它更好的信息来源了。HTTP/1.1 协议记录在 RFC 2616 中。MIME 类型的权威列表保存在：

<http://www.iana.org/assignments/media-types>

Kerrisk 是全面 Linux 编程的圣经，提供了现代网络编程的详细讨论[62]。关于计算机网络互联有大量很好的通用文献[65, 84, 114]。伟大的科技作家 W. Richard Stevens 编写了一系列相关的经典文献，如高级 Unix 编程[111]、因特网协议[109, 120, 107]，以及 Unix 网络编程[108, 110]。认真学习 Unix 系统编程的学生会想要研究所有这些内容。不幸的是，Stevens 在 1999 年 9 月 1 日逝世。我们会永远记住他的贡献。

家庭作业

- 11.6 A. 修改 TINY 使得它会原样返回每个请求行和请求报头。

B. 使用你喜欢的浏览器向 TINY 发送一个对静态内容的请求。把 TINY 的输出记录到一个文件中。

C. 检查 TINY 的输出，确定你的浏览器使用的 HTTP 的版本。

D. 参考 RFC 2616 中的 HTTP/1.1 标准，确定你的浏览器的 HTTP 请求中每个报头的含义。你可以从 www.rfc-editor.org/rfc.html 获得 RFC 2616。
- 11.7 扩展 TINY，使得它可以提供 MPG 视频文件。用一个真正的浏览器来检验你的工作。

- ** 11.8 修改 TINY, 使得它在 SIGCHLD 处理程序中回收操作系统分配给 CGI 子进程的资源, 而不是显式地等待它们终止。
- ** 11.9 修改 TINY, 使得当它服务静态内容时, 使用 malloc、rio_readn 和 rio_writen, 而不是 mmap 和 rio_writen 来复制被请求文件到已连接描述符。
- ** 11.10 A. 写出图 11-27 中 CGI adder 函数的 HTML 表单。你的表单应该包括两个文本框, 用户将需要相加的两个数字填在这两个文本框中。你的表单应该使用 GET 方法请求内容。
B. 用这样的方法来检查你的程序: 使用一个真正的浏览器向 TINY 请求表单, 向 TINY 提交填写好的表单, 然后显示 adder 生成的动态内容。
- ** 11.11 扩展 TINY, 以支持 HTTP HEAD 方法。使用 TELNET 作为 Web 客户端来验证你的工作。
- ** 11.12 扩展 TINY, 使得它服务以 HTTP POST 方式请求的动态内容。用你喜欢的 Web 浏览器来验证你的工作。
- ** 11.13 修改 TINY, 使得它可以干净地处理(而不是终止)在 write 函数试图写一个过早关闭的连接时发生的 SIGPIPE 信号和 EPIPE 错误。

练习题答案

11.1

十六进制地址	点分十进制地址
0x0	0.0.0.0
0xffffffff	255.255.255.255
0x7f000001	127.0.0.1
0xcdbca079	205.188.160.121
0x400c950d	64.12.149.13
0xcdbc9217	205.188.146.23

11.2

```

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct in_addr inaddr; /* Address in network byte order */
6      uint32_t addr;         /* Address in host byte order */
7      char buf[MAXBUF];     /* Buffer for dotted-decimal string */
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <hex number>\n", argv[0]);
11         exit(0);
12     }
13     sscanf(argv[1], "%x", &addr);
14     inaddr.s_addr = htonl(addr);
15
16     if (!inet_ntop(AF_INET, &inaddr, buf, MAXBUF))
17         unix_error("inet_ntop");
18     printf("%s\n", buf);
19
20     exit(0);
21 }

```

code/netp/hex2dd.c

11.3

```

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {

```

code/netp/dd2hex.c